Lab 3: Rendering

王唐欣宇 — 2400017808

November 13, 2025

1 Task 1: Phong Illumination

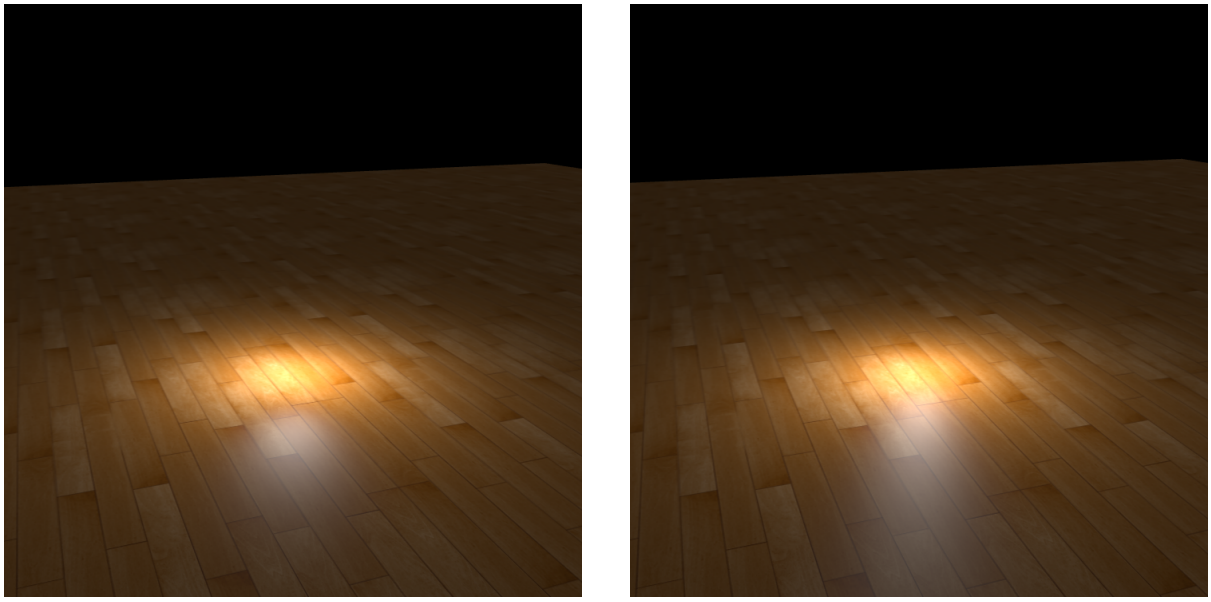
The Phong Illumination Model is given by:

$$I = k_a I_a + \sum_l (k_d (L \cdot N) I_l + k_s (R \cdot V)^n I_l)$$

The Blinn-Phong Illumination Model is given by:

$$I = k_a I_a + \sum_l (k_d (L \cdot N) I_l + k_s (H \cdot N)^n I_l)$$

I implemented both models in the shader directly, according to the formulas above. The results are shown in Figure 1.



(a) Phong Result. (b) Blinn-Phong Result.

Figure 1: The Results of Task 1.

Question 1

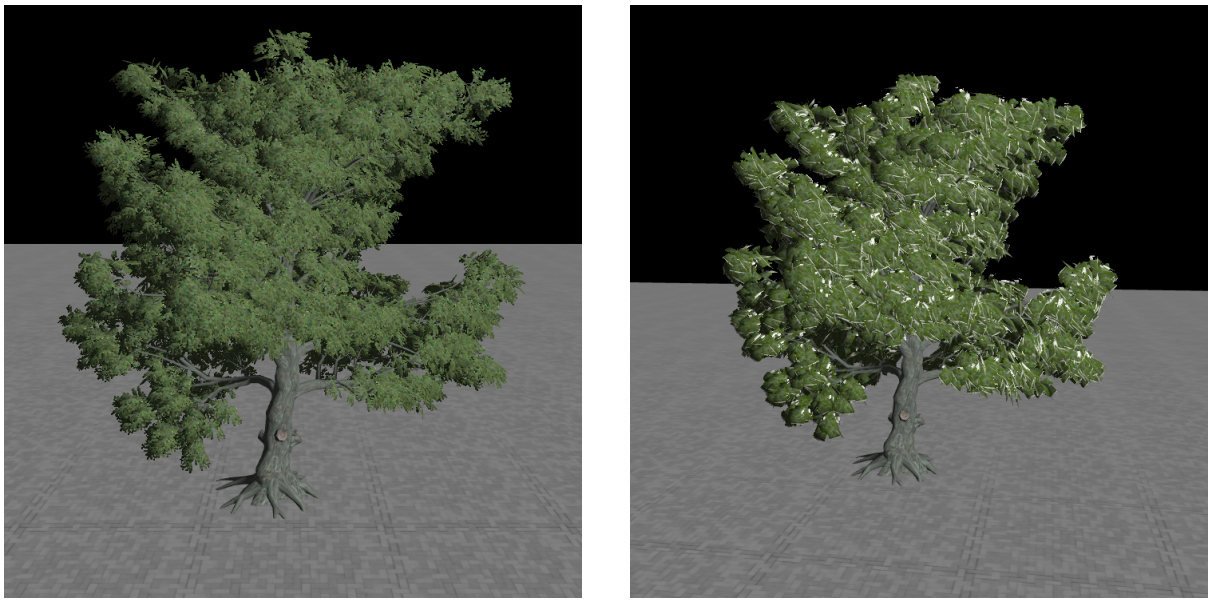
Relationship between the vertex shader and the fragment shader The vertex shader provides key per-vertex information to the fragment shader. It processes each vertex individually and outputs attributes such as position, color, and texture coordinates that describe the shape and appearance of the primitive.

How the outputs are passed to the fragment shader These outputs are automatically processed by the GPU during the rasterization stage, where they are interpolated across the surface of each primitive. In this way, each fragment (pixel) receives interpolated values based on its position within the primitive. The fragment shader then uses these interpolated inputs to compute the final color and other per-pixel properties for rendering.

Question 2

The statement `if (diffuseFactor.a < 0.2) discard;` means that if the alpha component of the diffuse factor is less than 0.2, the fragment will be discarded and not rendered. In other words, transparent regions of the texture will not contribute to the final image, which effectively removes the fully and nearly transparent parts of the white-oak texture and preserves the correct leaf shapes.

If this condition is changed to `if (diffuseFactor.a == 0.0) discard;`, only fragments with an alpha value exactly equal to 0.0 will be discarded. In practice, due to texture filtering, compression, and interpolation, very few texels have an alpha value that is precisely zero; pixels at the leaf boundaries usually have small but non-zero alpha values (e.g., $0.05-0.2$). These fragments are therefore not discarded and get rendered as semi-transparent pixels. As a result, the intended transparent regions around the leaves become faintly visible, producing a noticeable outline. Consequently, the white-oak texture appears incorrect, as the leaves lose their sharp silhouettes and the edges look partially translucent (see Figure 2).



(a) Proper Result. (b) Error Result.

Figure 2: The Difference of Alpha Testing.

Bonus Part

This part is for bonus 1.5 points.

I implemented the normal perturbation to achieve the bump mapping effect, following the method described in Mikkelsen *Bump Mapping Unparametrized Surfaces on the GPU*.

The position derivatives are computed as

$$\mathbf{p}_x = \frac{\partial \mathbf{p}}{\partial x}, \quad \mathbf{p}_y = \frac{\partial \mathbf{p}}{\partial y}.$$

Auxiliary vectors are defined by

$$\mathbf{r}_1 = \mathbf{p}_y \times \mathbf{n}, \quad \mathbf{r}_2 = \mathbf{n} \times \mathbf{p}_x, \quad \det = \mathbf{p}_x \cdot \mathbf{r}_1.$$

The height map samples are

$$H_{ll} = H(u, v), \quad H_{lr} = H(u + \Delta u_x, v + \Delta v_x), \quad H_{ul} = H(u + \Delta u_y, v + \Delta v_y).$$

The surface gradient is computed as

$$\nabla_s H = \text{sign}(\det)\big[(H_{lr} - H_{ll})\,\mathbf{r}_1 + (H_{ul} - H_{ll})\,\mathbf{r}_2\big],$$

and the perturbed normal is

$$\mathbf{n}' = \frac{|\det|\,\mathbf{n} - \nabla_s H}{\|\,|\det|\,\mathbf{n} - \nabla_s H\,\|}.$$

Finally, the blended normal is

$$\mathbf{n}_{\text{final}} = \frac{(1-\alpha)\mathbf{n} + \alpha\mathbf{n}'}{\|(1-\alpha)\mathbf{n} + \alpha\mathbf{n}'\|}, \quad \alpha = u_{\text{BumpMappingBlend}}.$$

This implementation successfully produces detailed surface variation without increasing the mesh complexity.

The results are shown in Figure 3.

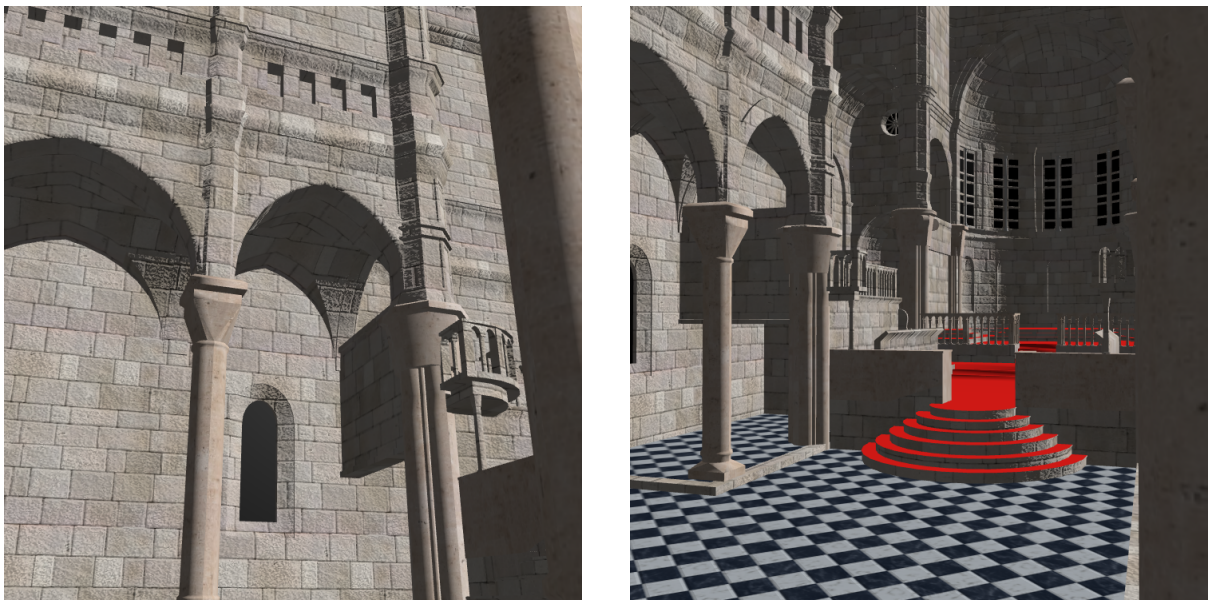


Figure 3: Bump Mapping Results.

2 Task 2: Environment Mapping

I implemented environment mapping using cube maps. To avoid translation artifacts, I extract the rotation part of the view matrix to get the texture coordinates for sampling the cube map. The results are shown in Figure 4.

The results are shown as follows:

(a) Teapot Result.

(b) Bunny Result.

Figure 4: The Results of Task 2.

3 Task 3: Non-Photorealistic Rendering

I implemented the basic cel-shading algorithm as described in the PPT.

`u_ToneLevels` can be adjusted to change the number of discrete shading levels, and I used 3 levels in the final rendering. The results are shown in Figure 5.

The results are shown as follows:

Question 1

`CaseNonPhoto.cpp` uses `glCullFace(GL_FRONT)` and `glEnable(GL_CULL_FACE)` to render only back faces first. Then it uses `glCullFace(GL_BACK)` and `glEnable(GL_CULL_FACE)` to render only front faces with cel shading. This approach ensures that the outline is drawn behind the cel-shaded model, creating a clear and distinct outline effect.

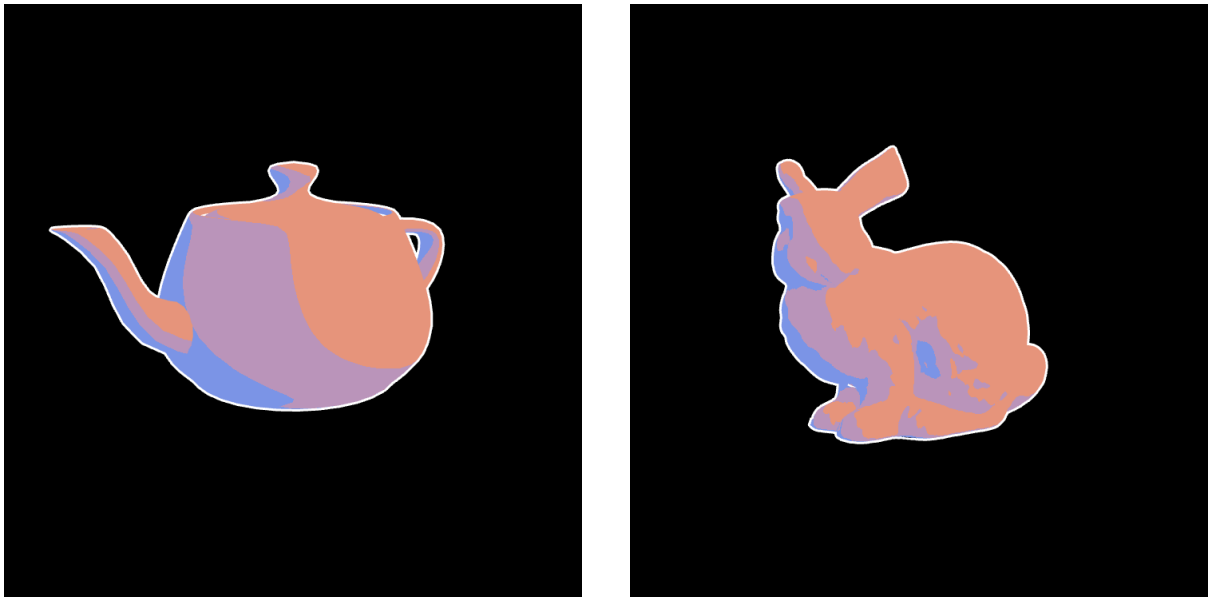
Question 2

We cannot simply offset a vertex along its normal in world space to generate an outline because this approach does not produce a consistent outline thickness in screen space. The apparent thickness of the outline would depend on the angle between the view direction and the surface normal: vertices whose normals point more toward or away from the camera would produce longer or shorter projected offsets on the screen. This results in outlines that are uneven and visually unappealing.

To achieve a constant, visually uniform outline thickness, we must perform the offset in screen space (or clip space). By computing the displacement after projection, we ensure that the outline width is measured in pixels on the screen, independent of the orientation or distance of the surface, which guarantees a consistent and stable non-photorealistic rendering effect.

4 Task 4: Shadow Mapping

The only code blank I need to fill is `closestDepth`. For a directional light, we can directly use `texture(u_ShadowMap, pos.xy).r` to get the proper depth. For a point light, we must sample



(a) Teapot Result. (b) Bunny Result.

Figure 5: The Results of Task 3.

it in a cube map and convert the depth from $[0, 1]$ back to world space.

The results are shown in Figure 6.

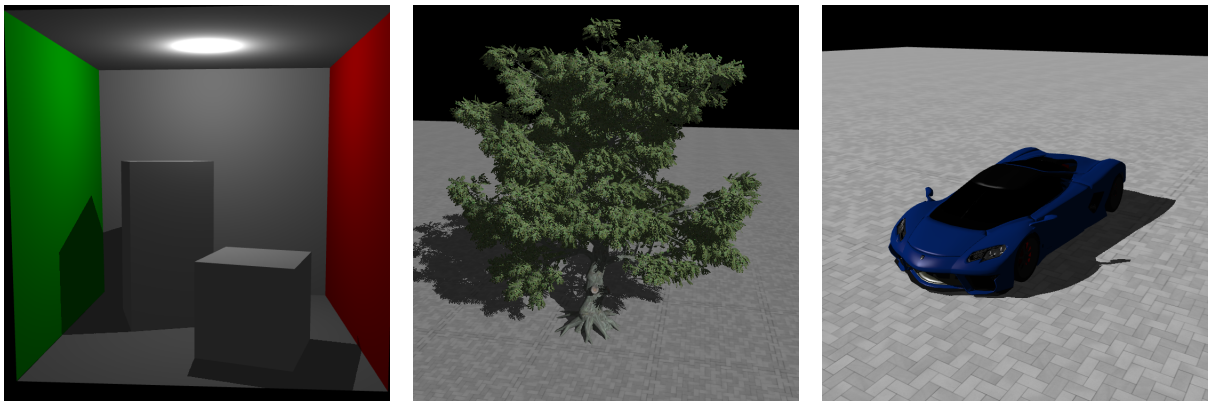


Figure 6: The Results of Task 4.

Question 1

The directional-light shadow uses orthographic projection to get the proper depth. The point-light shadow uses perspective projection to get the proper depth.

Question 2

The shadow depth is calculated independently in other pipelines, so we can use it directly in the shader without generating it again. The `shadowmap.vert` and `shadowmap.frag` shaders are used to generate the shadow map from the light's perspective.

5 Task 5: Whitted-Style Ray Tracing

I implemented a basic Whitted-style ray tracer that supports shadows, reflections, and refractions. I use the Möller–Trumbore algorithm to perform ray–triangle intersection tests.

Question

Rendering Principles: Rasterization projects 3D geometry onto 2D screens and approximates lighting per pixel, while ray tracing simulates light paths, including reflection, refraction, and shadows, for more physically accurate results.

Rendering Results: Rasterization is fast and suitable for real-time applications but simplifies shadows, reflections, and indirect lighting. Ray tracing produces more realistic, photorealistic effects at higher computational cost.

Similarities and Differences: Both methods compute surface colors and produce 2D images. Rasterization emphasizes efficiency, whereas ray tracing emphasizes physical realism. Intuitively, rasterization is "from objects to screen," ray tracing is "from light to screen."

Summary: Rasterization = fast, real-time, approximate; Ray tracing = accurate, realistic, computationally intensive. Modern graphics often combine both for a balance of speed and realism.

Bonus Part

This part is for bonus 2.5 points.

I use an AABB to accelerate the ray–triangle intersection tests, which is implemented in the `WTXYRayIntersector` struct. Instead of flattening all models' triangles into a big list, I build a BVH tree for each model, because I think this is a good initial spatial split.

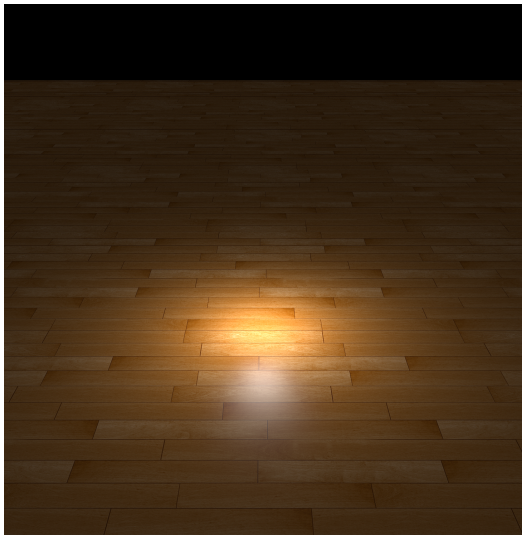
In experiments, I observed that the same triangle may appear in multiple models. I did not handle this case; I simply use the smaller model index to select the final triangle ID.

The rendering times are listed in Table 1, rendered at 1 sample per pixel with a maximum of 7 reflection bounces:

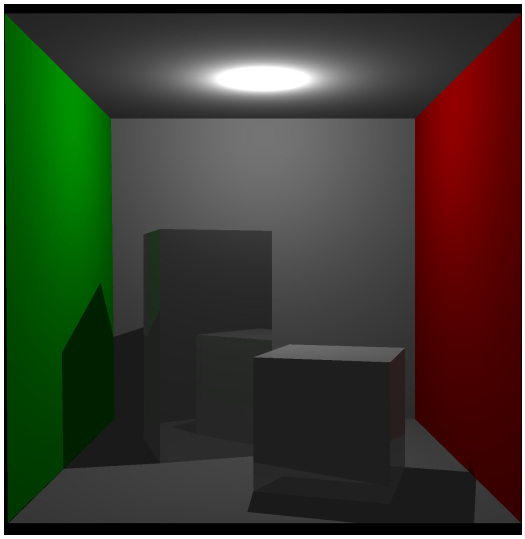
Scene	time (s)
floor	1
cornell_box	4
white_oak	430
sports_car	32
breakfast_room	117
sibenik	701
sponza	128

Table 1: Rendering Time

The results are shown in Figure 7.



(a) Floor

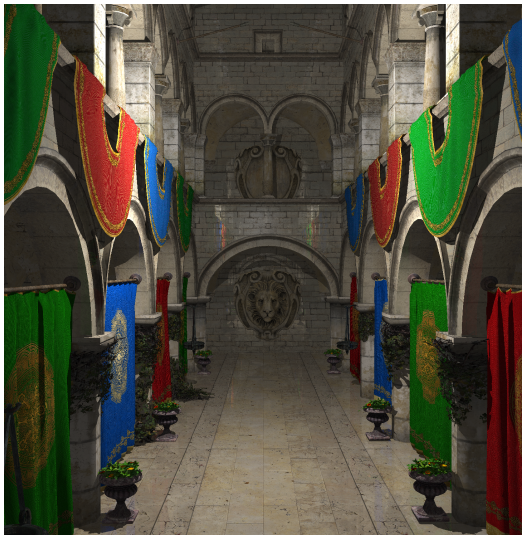


(b) Cornell Box

(c) White Oak

(d) Sports Car

(e) Breakfast Room



(f) Sponza

Figure 7: The Results of Task 5.